

Automatic Retry Manager

Financial consulting teams manage communication between internal services and downstream systems (authentication, billing, telemetry). These systems sometimes fail temporarily due to network issues, but retrying usually works. Instead of adding retry logic everywhere, you need to create an automatic retry mechanism that can be toggled on/off at runtime.

Implement a global `RetryManager` class and a `@retryable` decorator with type hints that provides robust failure recovery. The decorator should automatically retry failed operations based on the global state and a per-function retry limit.

Example 1 Input:

```
SET_ENABLED true CALL 3 1
```

Output:

```
Attempt 1 failed: Transient Error Success after 2 attempts
```

Explanation: The retry mechanism is enabled. The task is called with `maxRetries=3` and is set to fail once. The first attempt fails, triggering the "Attempt 1 failed" message. The second attempt succeeds, triggering the "Success after 2 attempts" summary.

Example 2 Input:

```
SET_ENABLED false CALL 3 1
```

Output:

```
Retry disabled
```

Explanation: The retry mechanism is globally disabled. The decorator prints "Retry disabled" and executes the function only once. Since the function fails on its first attempt and no retries are allowed, it raises the exception normally (the driver captures this behavior).

Function Parameter:

- `maxRetries` (int): The maximum number of attempts including the initial one.

- `flag` (bool): Used in `setEnabled` to toggle the global state.

Returns

- The original function's return value on success.
- Re-raises the last exception if all retries fail.

Constraints

- $1 \leq \text{maxRetries} \leq 10$
- The decorator must use `functools.wraps` to preserve function metadata.
- Global state must be stored in a private class variable `_enabled`.

Input Format for Custom Testing Input from stdin will be processed as follows: The first line contains the global state command `SET_ENABLED` followed by `true` or `false`. Subsequent lines contain `CALL` followed by `maxRetries` and `numFails`.

Sample Case 0 Sample Input

```
SET_ENABLED true CALL 2 2
```

Sample Output

```
Attempt 1 failed: Transient Error Attempt 2 failed: Transient Error  
ERROR: all retries failed
```

Explanation The task is set to fail twice, but `maxRetries` is only 2. Both attempts fail, and the final "ERROR: all retries failed" message is printed before the exception is re-raised.